

Homework 1: Linear Regression and Neural Network Regression

Step 1. Data

- 1) There are 3047 samples in this dataset
- 2) Since the target label *TARGET_deathRate* is continuous, this is a regression problem. It's trying to predict cancer mortality rates based on personal information related features such as education, income, age, race, as well as demographic features like population, poverty percentage and study per county, and several health related features such as average number of reported cases annually, average number of deaths per year.
- 3)

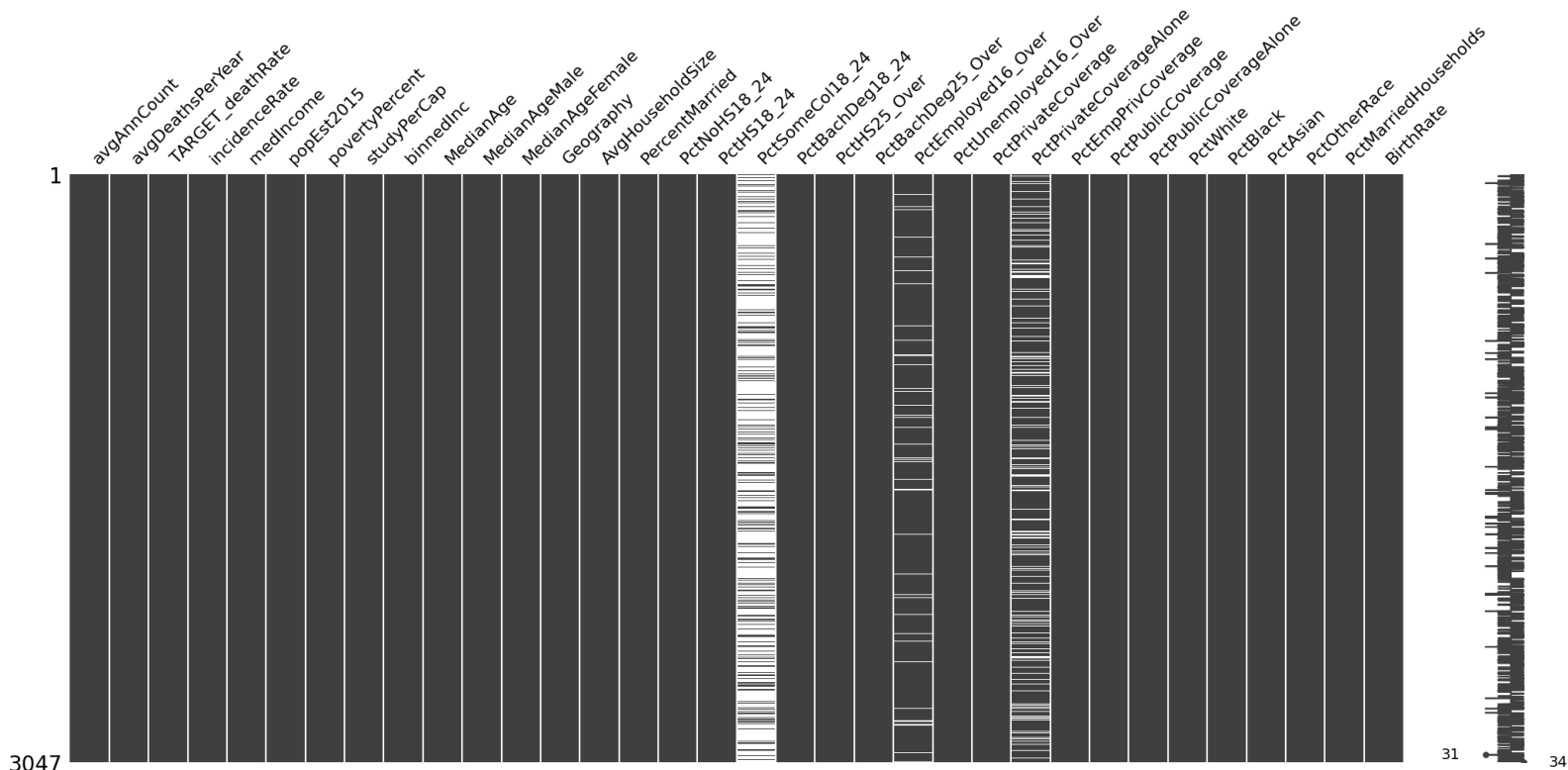
avgAnnCount	MIN = 6.0	MAX = 38150.0
avgDeathsPerYear	MIN = 3.0	MAX = 14010.0
TARGET_deathRate	MIN = 59.7	MAX = 362.8
incidenceRate	MIN = 201.3	MAX = 1206.9
medIncome	MIN = 22640.0	MAX = 125635.0
popEst2015	MIN = 827.0	MAX = 10170292.0
povertyPercent	MIN = 3.2	MAX = 47.4
studyPerCap	MIN = 0.0	MAX = 9762.308998
MedianAge	MIN = 22.3	MAX = 624.0
MedianAgeMale	MIN = 22.4	MAX = 64.7
MedianAgeFemale	MIN = 22.3	MAX = 65.7
AvgHouseholdSize	MIN = 0.0221	MAX = 3.97
PercentMarried	MIN = 23.1	MAX = 72.5
PctNoHS18_24	MIN = 0.0	MAX = 64.1
PctHS18_24	MIN = 0.0	MAX = 72.5
PctSomeCol18_24	MIN = 7.1	MAX = 79.0
PctBachDeg18_24	MIN = 0.0	MAX = 51.8
PctHS25_Over	MIN = 7.5	MAX = 54.8
PctBachDeg25_Over	MIN = 2.5	MAX = 42.2
PctEmployed16_Over	MIN = 17.6	MAX = 80.1
PctUnemployed16_Over	MIN = 0.4	MAX = 29.4
PctPrivateCoverage	MIN = 22.3	MAX = 92.3
PctPrivateCoverageAlone	MIN = 15.7	MAX = 78.9
PctEmpPrivCoverage	MIN = 13.5	MAX = 70.7
PctPublicCoverage	MIN = 11.2	MAX = 65.1
PctPublicCoverageAlone	MIN = 2.6	MAX = 46.6
PctWhite	MIN = 10.1991551	MAX = 100.0
PctBlack	MIN = 0.0	MAX = 85.94779858
PctAsian	MIN = 0.0	MAX = 42.61942454
PctOtherRace	MIN = 0.0	MAX = 41.93025142
PctMarriedHouseholds	MIN = 22.99248989	MAX = 78.07539683
BirthRate	MIN = 0.0	MAX = 21.32616487

- 4) There are 33 features (excluding target label), including 31 numerical features and 2 categorical features (*Geography* and *binnedInc*)
- 5) Only 3 features have missing values, and it's noticeable that *PctSomeCol18_24* has roughly 75% missing values of the whole dataset.

```
df.isna().sum().sort_values(ascending=False)
```

PctSomeCol18_24	2285	PctSomeCol18_24	74.991795
PctPrivateCoverageAlone	609	PctPrivateCoverageAlone	19.986872
PctEmployed16_Over	152	PctEmployed16_Over	4.988513
PctEmpPrivCoverage	0	PctEmpPrivCoverage	0.000000
PctHS25_Over	0	PctHS25_Over	0.000000
PctBachDeg25_Over	0	PctBachDeg25_Over	0.000000
PctUnemployed16_Over	0	PctUnemployed16_Over	0.000000
PctPrivateCoverage	0	PctPrivateCoverage	0.000000
PctPublicCoverage	0	PctPublicCoverage	0.000000
avgDeathsPerYear	0	avgDeathsPerYear	0.000000
PctPublicCoverageAlone	0	PctPublicCoverageAlone	0.000000
PctWhite	0	PctWhite	0.000000
PctBlack	0	PctBlack	0.000000
PctAsian	0	PctAsian	0.000000
PctOtherRace	0	PctOtherRace	0.000000
PctMarriedHouseholds	0	PctMarriedHouseholds	0.000000
PctBachDeg18_24	0	PctBachDeg18_24	0.000000
avgAnnCount	0	avgAnnCount	0.000000
PctHS18_24	0	PctHS18_24	0.000000
PctNoHS18_24	0	PctNoHS18_24	0.000000
PercentMarried	0	PercentMarried	0.000000
AvgHouseholdSize	0	AvgHouseholdSize	0.000000
Geography	0	Geography	0.000000
MedianAgeFemale	0	MedianAgeFemale	0.000000
MedianAgeMale	0	MedianAgeMale	0.000000
MedianAge	0	MedianAge	0.000000
binnedInc	0	binnedInc	0.000000
studyPerCap	0	studyPerCap	0.000000
povertyPercent	0	povertyPercent	0.000000
popEst2015	0	popEst2015	0.000000
medIncome	0	medIncome	0.000000
incidenceRate	0	incidenceRate	0.000000
TARGET_deathRate	0	TARGET_deathRate	0.000000
BirthRate	0	BirthRate	0.000000
dtype: int64		dtype: float64	

Below is nullity matrix to visualize where the missing values locate in each feature:



- 6) The label of this dataset is *TARGET_deathRate* (cancer mortality rate)
- 7) For this dataset, I used 80% for training data, 10% for validation and 10% for the testset.
- 8) For preprocessing, since *PctSomeCo18_24* and *PctPrivateCoverageAlone* features contain ~75% and ~20% of missing values, it's better to just drop the entire columns so that the model won't try to learn noises during training. Trying to impute, on the other hand, will risk making those features contribute noises to the model or the model may be overfit to those fake patterns, which can cause poor generalization. Additionally, since *PctEmployed16_Over* feature only contains less than 5% missing values, it's safe to impute with the median to preserve this feature's information.

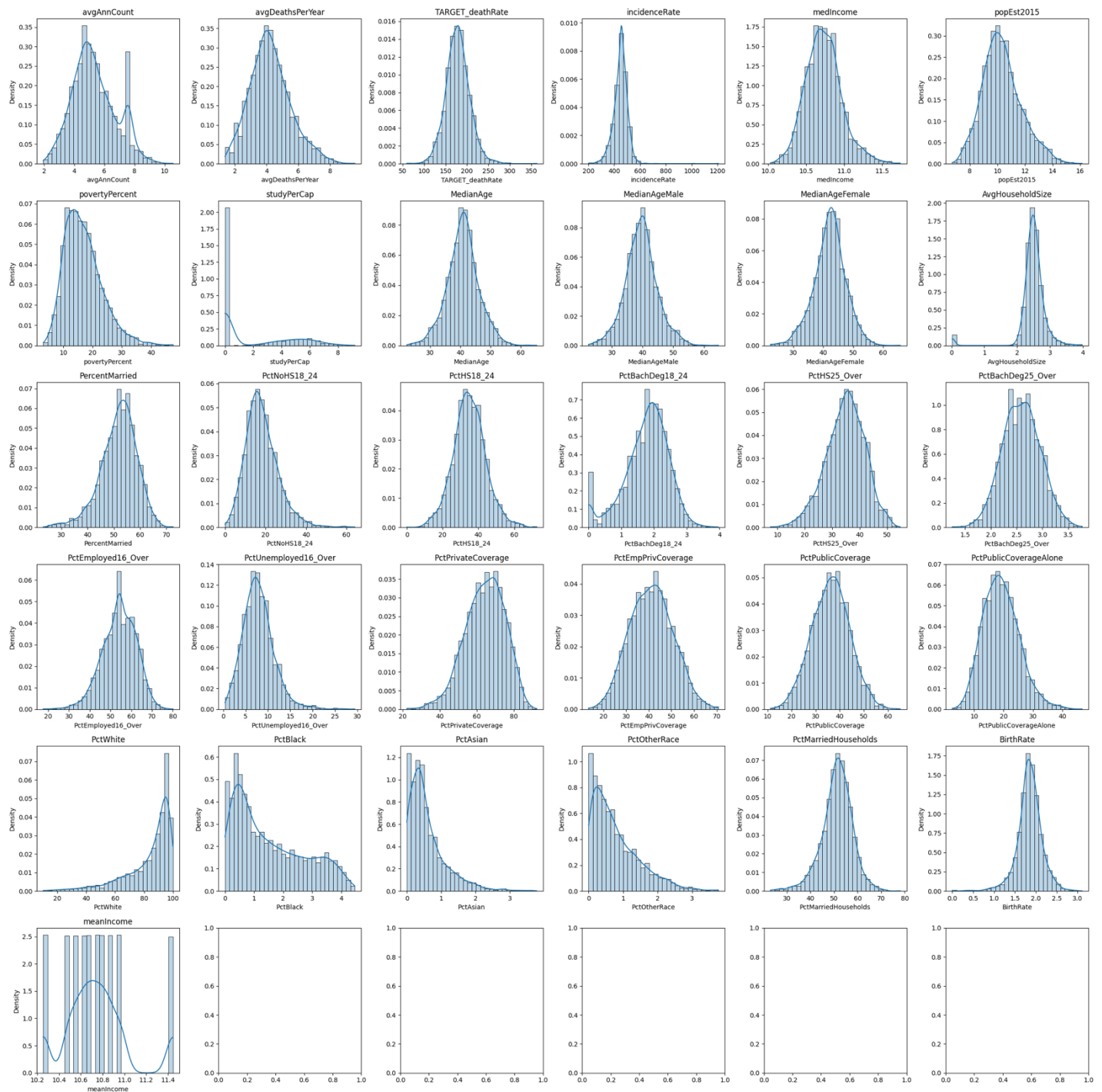
Next, while exploring the dataset, I realized an interesting part that *MedianAge* feature has a maximum value of 624, which is impossible. Investigating further, I saw some of the samples has *MedianAge* values over 120. My assumption was that the value may be mistakenly multiplied by 10

when collecting the dataset. Therefore, I modified those samples by dividing the *MedianAge* by 10.

For categorical features, since *binnedInc* contains the range of income, it would be useful to extract the mean income by calculating the average of the 2 values min and max from that feature so that the model can also learn from mean rather than just the median in *medIncome*. Moreover, for *Geography*, since all the values are unique, it won't be helpful if we apply one hot encoding to this feature since it will create a giant sparse matrix. Therefore, my first approach was trying to extract the state from the feature and apply one hot encoding so that only 51 features (states) are added. However, it turned out that the model didn't improve, so I also decided to drop *Geography* feature in the end.

Also in EDA, I observed that most of the feature distribution is skewed, such as *PctBlack*, *PctAsian*, which has few values having extremely large values while the majority has small values, so I applied log transformation to compress those large values and expand small ones, making the distribution closer to normal in order to help improve stability when training model. In this case, I only applied log transformation to features having skewness score > 1 .

Below are the distributions after transformation:



Finally, I splitted the dataset into 3 sets, training set (80%), validation set (10%) and test set (10%), and applied normalization to the datasets to scale the features down so the training phase can be more stable. Here, I used Robust Scaler (based on median and IQR) since it's more robust to outliers in this dataset.

Step 2. Model

Here I trained linear regression and deep neural networks with these architectures. Below table is the model architecture choices with their corresponding best r2 scores:

Model	Test R-squared
Linear regression	0.78
DNN-16	0.81
DNN-30-8	0.88
DNN-30-16-8	0.81
DNN-30-16-8-4	0.79
DNN-64-30-16-8	0.77
DNN-128-64-32	0.74

When training, I used batch size of 128, leaky relu activation, and I also applied batch normalization, gradient clipping, and early stopping techniques to help avoid gradient exploding, stable training and better generalization.

Additionally, I tried adding other techniques like L2 regularization, dropout, however, with this dataset, they didn't seem to work, so I disabled them.

Step 3: Objective

Mean Squared Error (MSE) is the loss function I used to train the models

Step 4: Optimization

For optimization, I used SGD with momentum of 0.9 to help the model converge a little faster while training

Step 5: Model selection

Model	LR: 0.1 (R ²)	LR: 0.01 (R ²)	LR: 0.001 (R ²)	LR: 0.0001 (R ²)
Linear Regression	0.78	0.78	0.78	0.78
DNN-16	0.73	0.79	0.81	0.68
DNN-30-8	0.80	0.84	0.88	0.55
DNN-30-16-8	0.39	0.81	0.80	0.57
DNN-30-16-8-4	0.59	0.79	0.51	0.52
DNN-64-30-16-8	0.69	0.77	0.40	0.47
DNN-128-64-32	-0.07	0.74	0.66	0.57

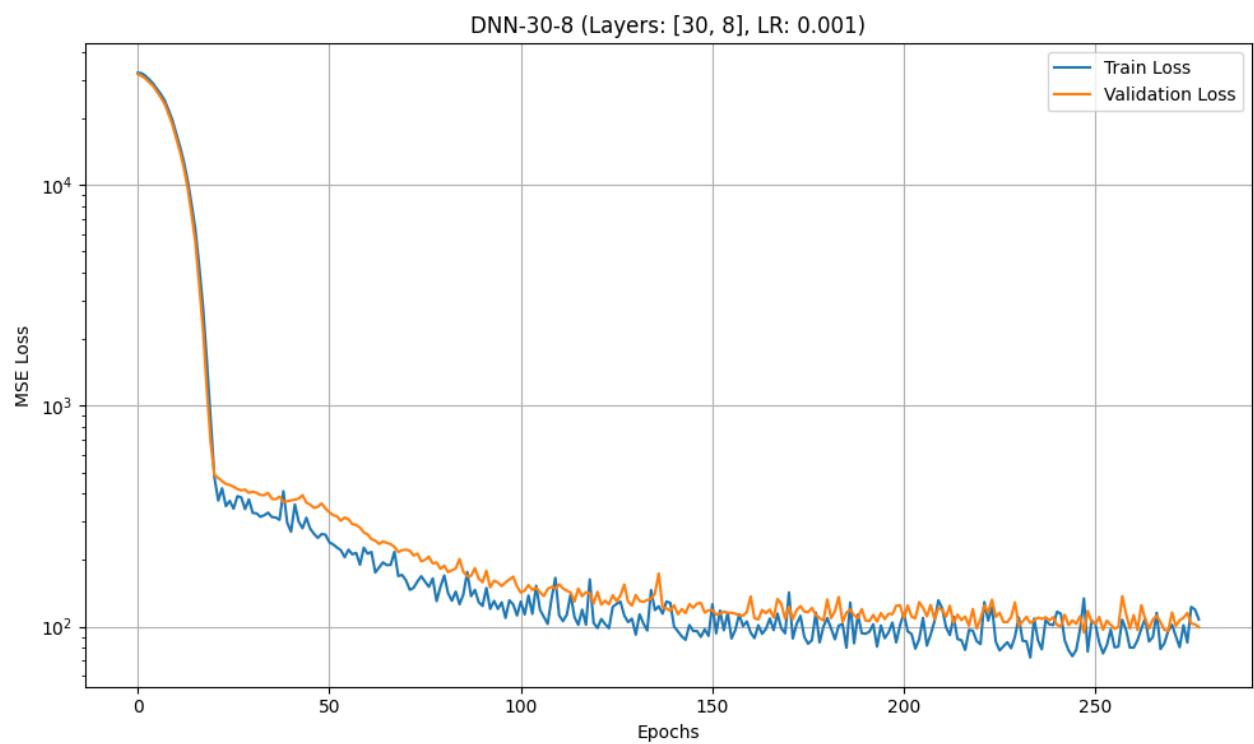
* For linear regression, I used sklearn so it isn't dependent on the learning rate.
Therefore, r^2 is the same for all learning rates

* I used **green** as the baseline for other models to compare, **red** for best $r^2 >$ linear regression, and **blue** otherwise

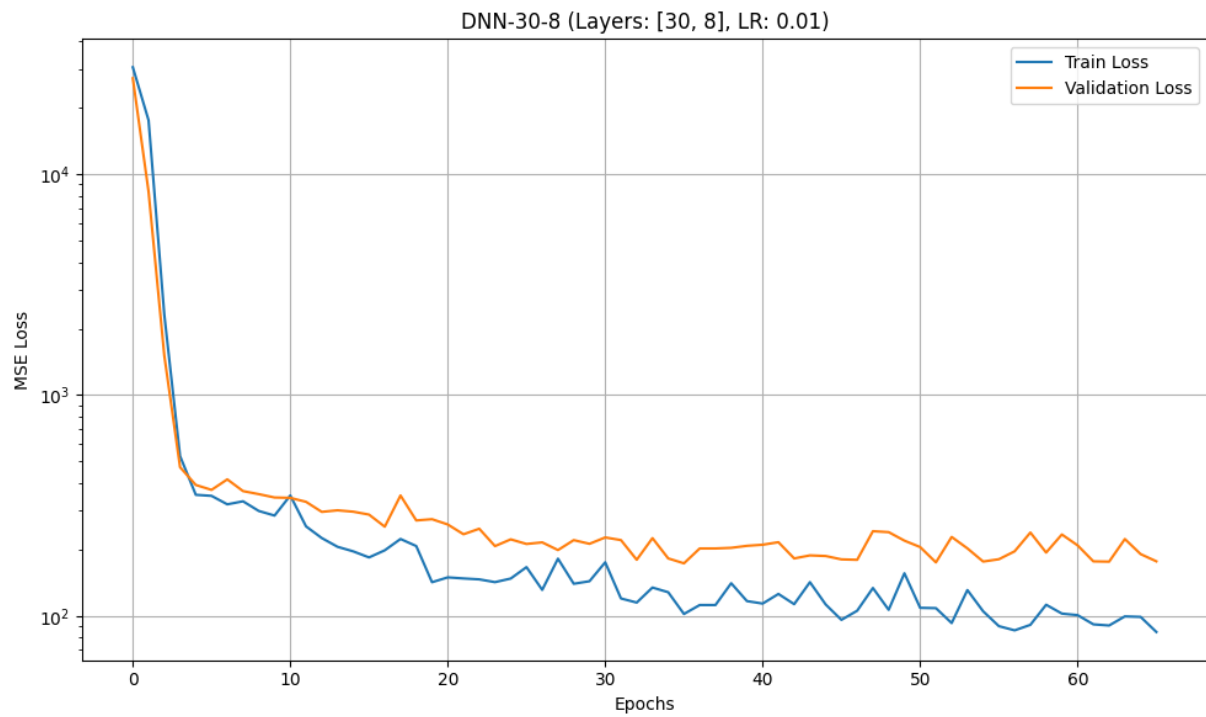
Step 6: Model performance

Below are MSE losses over epoch for each best model, noted that the losses are log transformed when visualizing:

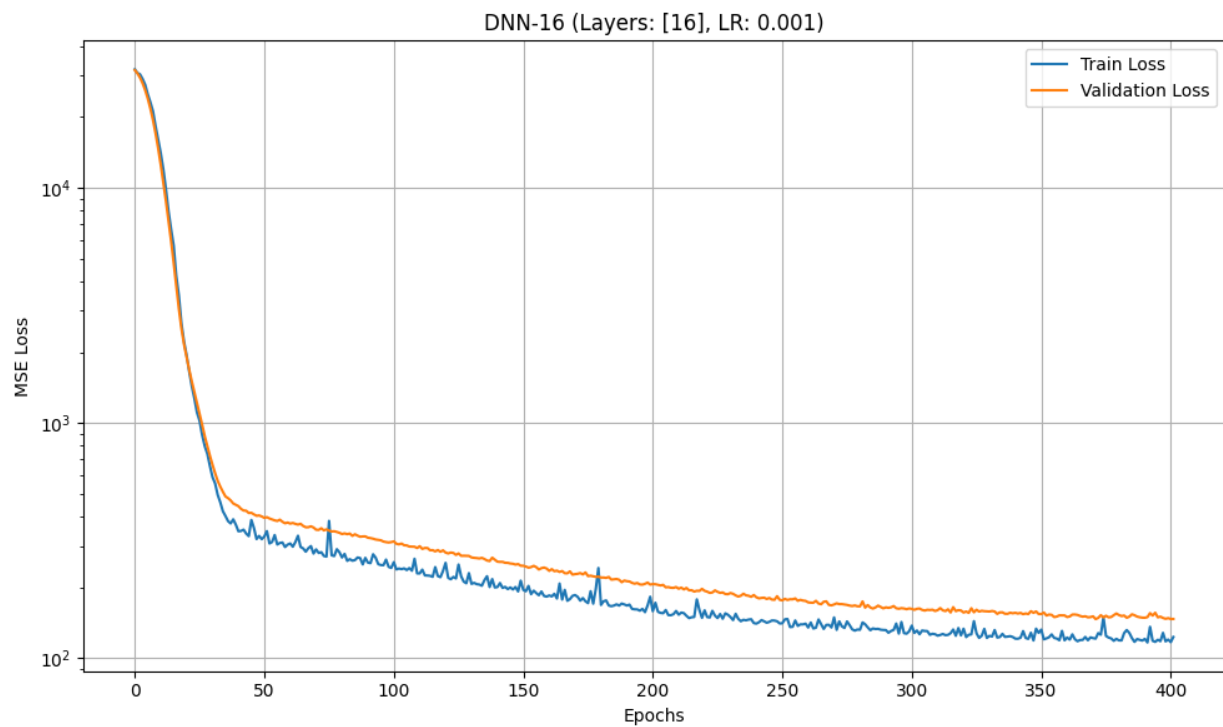
- DNN-30-8 with learning rate 0.001 ($r^2 = 0.88$)



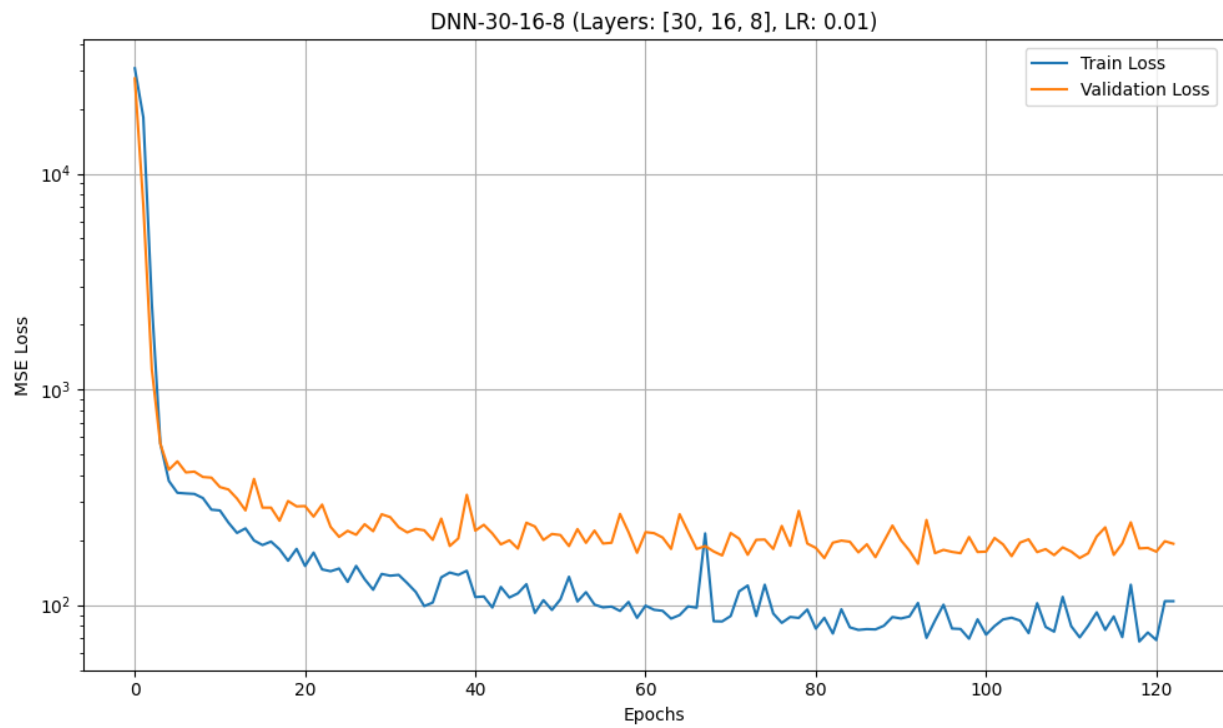
- DNN-30-8 with learning rate 0.01 ($r^2 = 0.84$)



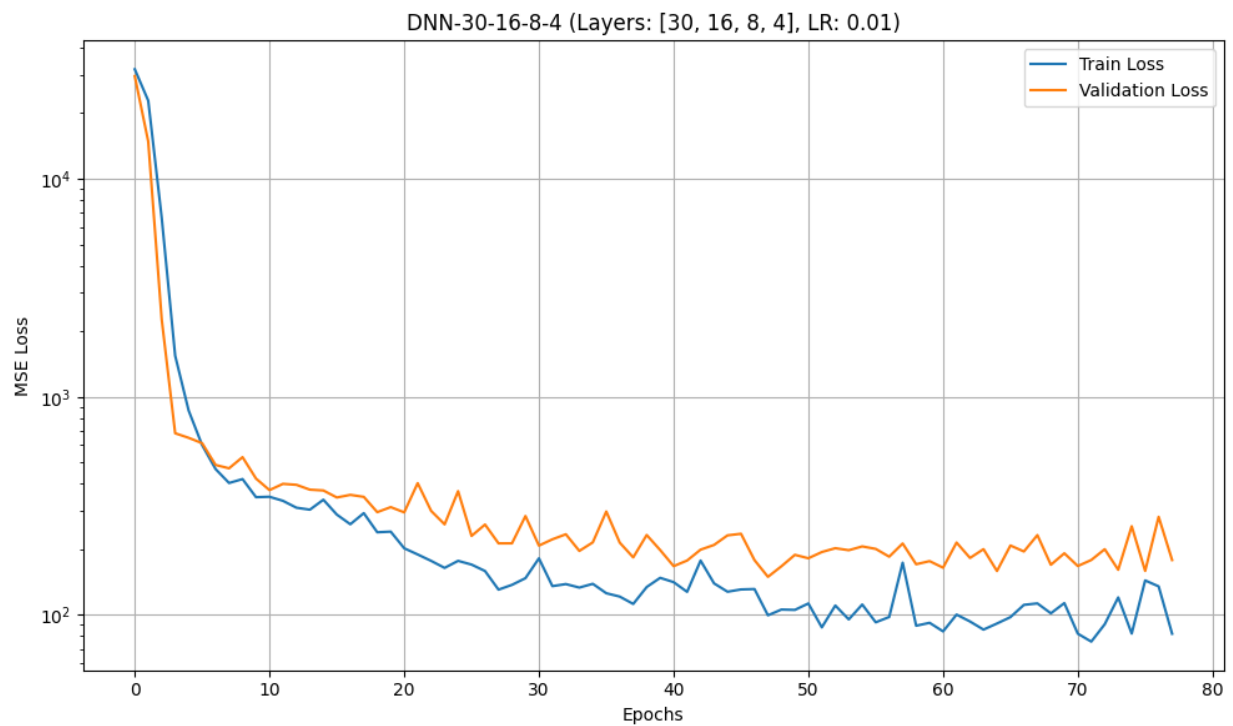
- DNN-16 with learning rate 0.001 ($r^2 = 0.81$)



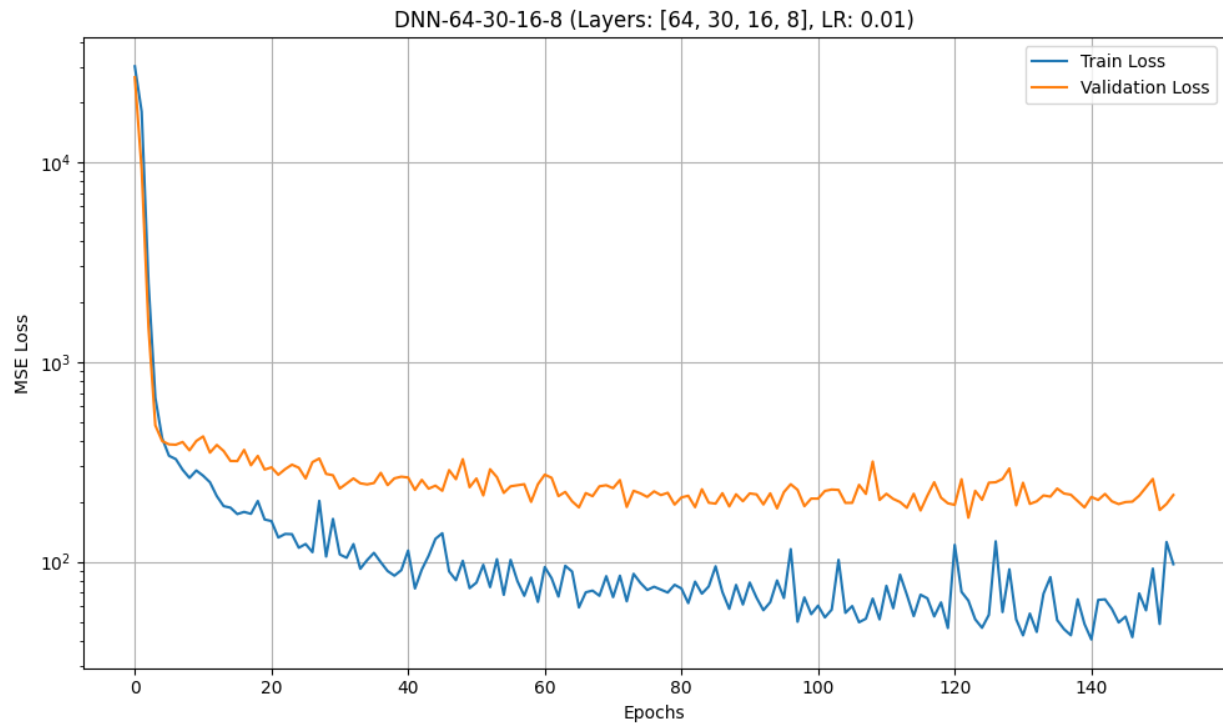
- DNN-30-16-8 with learning rate 0.01 ($r^2 = 0.81$)



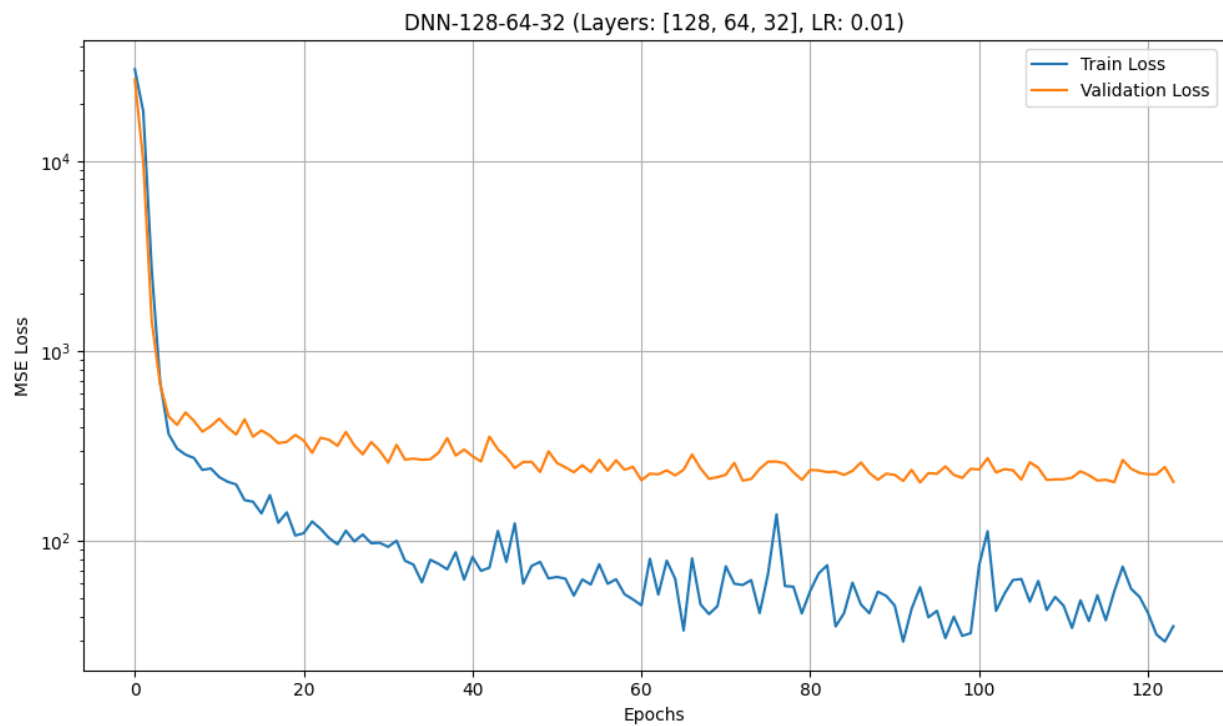
- DNN-30-16-8-4 with learning rate 0.01 ($r^2 = 0.79$)



- DNN-64-30-16-8 (custom model) with learning rate 0.01 ($r^2 = 0.77$)

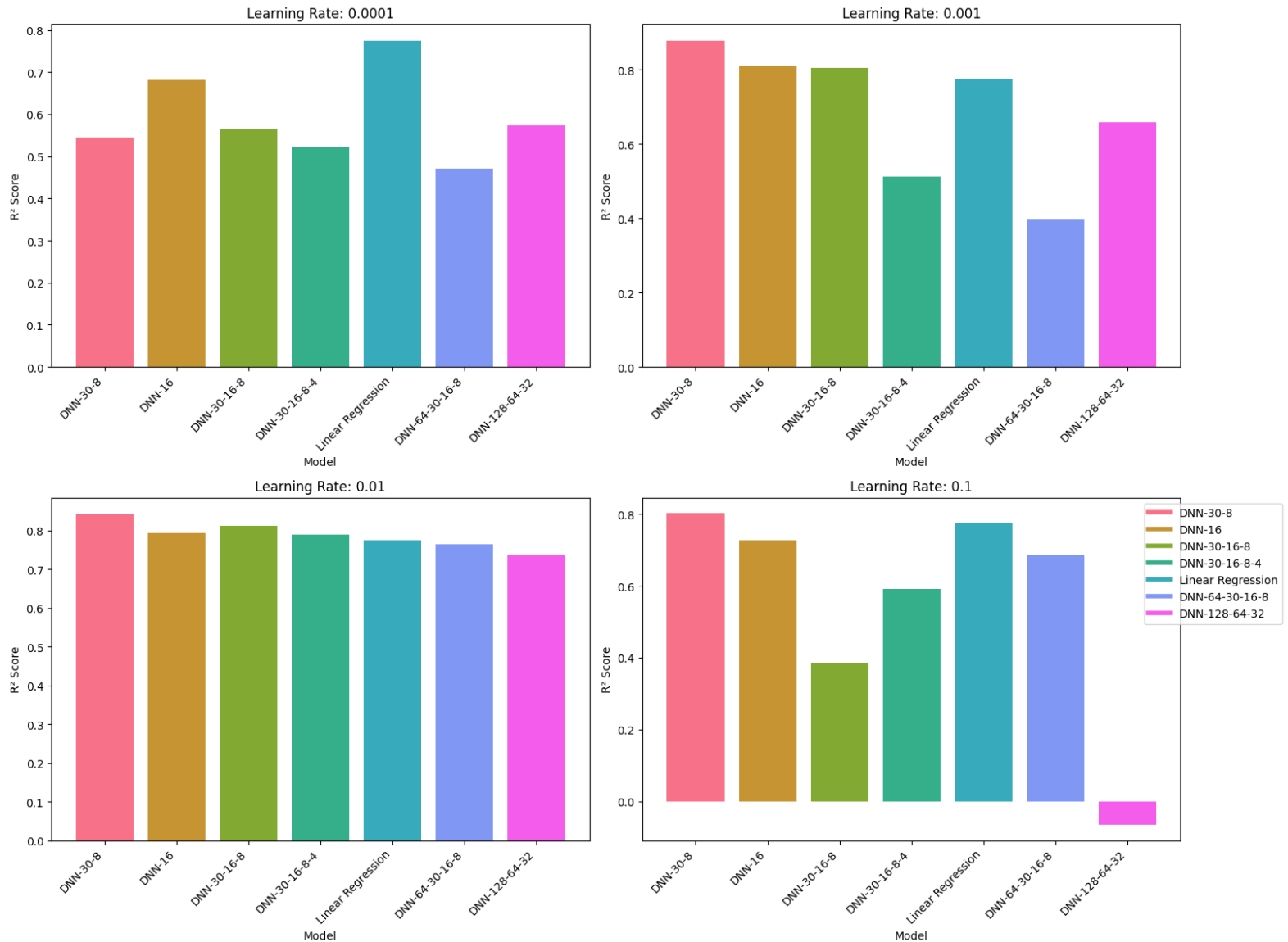


- DNN-128-64-32 with learning rate 0.01 ($r^2 = 0.74$)



	model	learning rate	r2
1	DNN-30-8	0.001	0.8790460824966431
2	DNN-30-8	0.01	0.8426064848899841
3	DNN-16	0.001	0.812458336353302
4	DNN-30-16-8	0.01	0.8123619556427002
5	DNN-30-16-8	0.001	0.8043397665023804
6	DNN-30-8	0.1	0.8022156953811646
7	DNN-16	0.01	0.7939363121986389
8	DNN-30-16-8-4	0.01	0.7892199754714966
9	Linear Regression	0.0001	0.7750838398933411
10	Linear Regression	0.1	0.7750838398933411
11	Linear Regression	0.01	0.7750838398933411
12	Linear Regression	0.001	0.7750838398933411
13	DNN-64-30-16-8	0.01	0.765041172504425
14	DNN-128-64-32	0.01	0.7362129092216492
15	DNN-16	0.1	0.7270095348358154
16	DNN-64-30-16-8	0.1	0.687917947769165
17	DNN-16	0.0001	0.68216872215271
18	DNN-128-64-32	0.001	0.6588846445083618
19	DNN-30-16-8-4	0.1	0.5911782383918762
20	DNN-128-64-32	0.0001	0.5735647678375244
21	DNN-30-16-8	0.0001	0.5660535097122192
22	DNN-30-8	0.0001	0.5456589460372925
23	DNN-30-16-8-4	0.0001	0.5226719975471497
24	DNN-30-16-8-4	0.001	0.513170599937439
25	DNN-64-30-16-8	0.0001	0.4717760682106018
26	DNN-64-30-16-8	0.001	0.3979495763778686
27	DNN-30-16-8	0.1	0.3850288987159729
28	DNN-128-64-32	0.1	-0.0655398368835449

Below plot helps visualize the performance for comparison:



It's noticeable that increasing model's capacity for this dataset decreases the performance a lot. The 2 customized model I added actually performed worse than linear regression.